and flags. A positional argument is a value provided based on its spot on the command-line and is typically a piece of data or a filter on data. A flag is an optional argument that changes the behavior of the command.

For example, in Unix, the `zip` command creates a compressed archive. It takes two or more positional arguments. The first argument is the name of the archive to be created, the second and any subsequent arguments are the names of the files to put in the archive. If you want to know more about what `zip` is doing, you can add the `--verbose` flag, and it will print out more information. Figure 9.1 shows the parts of the `zip` command and its arguments.



Figure 9.1 Positional and flag arguments to the `zip` command

In Unix systems, flags begin with either a dash (`-`) or a double-dash (`--`). Typically, the double-dash format is a readable word, while the single dash is an abbreviation. For example, most Unix commands support `-h` and `--help` as optional flags, both causing the command's help information to be printed to the terminal. With the `zip` command, `--verbose` can be shortened to `-v`. The `BaseCommand` class implements this flag for you out of the box.

> **NOTE** Normally, I'd write about a function's arguments, but when you're writing about command-line arguments and function arguments in the same sentence, that can be confusing. For the rest of this chapter, a function's arguments will be referred to as *parameters* to help differentiate.

All your script's arguments get passed as strings by the operating system to your program. The Python standard library has a module called `argparse`, which parses these strings and turns them into positional arguments and flags. Django's management commands wrap this very same library, so you have less code to write.

`argparse` references

The examples in this chapter don't require familiarity with `argparse`; you'll be fine if you haven't used it before. It's a quite complex library, with a lot of choices. To take full advantage of its power, you'll want to dig in a little deeper:

- Python's documentation: http://mng.bz/M9jE
- Python's `argparse` how-to: http://mng.bz/amax
- An in-depth tutorial: http://mng.bz/g7GZ